STACK & SCALE REMOTE INTERNSHIP 2026

WEEK 02 — COMPREHENSIVE PROJECT REPORT

SpendWise

Personal Budget & Expense Tracker

Offline-First Financial Management Engine Built with Flutter, Dart & SQLite

Project Specification	Submission Metadata Details
Internship Cohort	Stack & Scale Remote Internship 2026 — Cohort Evaluation
Project Title	SpendWise — Personal Budget & Expense Tracker
Evaluation Milestone	Week 02 Final Deliverable (Days 1 to 5 Complete)
Lead Developer	Nida Fatima
Project Duration	02 November 2026 – 06 November 2026
Application Framework	Flutter 3.x / Dart 3.x (Material 3 Fintech Theme)
Database Architecture	Offline-First Relational SQLite (sqflite / sqflite_common_ffi)
State Management	Native Flutter ChangeNotifier (Zero External Dependency Bloat)
Precision Currency	PKR Minor Units Standard (Integer Paisas / 1 PKR = 100 Paisa)
Target Audience	Students, Early-Career Professionals & Freelance Developers
Verification Status	37 / 37 Automated Tests Passing • 0 Flutter Analyze Issues

Formal Submission Certification

This technical report constitutes the official Week 02 project documentation for SpendWise. All screenshots, architectural schemas, source code snippets, database structures, and automated test outputs presented herein are authentic artifacts generated from the live Flutter runtime and verified against the internship requirements.

Section 2: Project Introduction

SpendWise is a production-grade, offline-first personal budget and expense tracking application engineered to help students, recent graduates, and early-career software professionals master their daily spending and monthly cash flow. Personal financial discipline in the modern era is frequently hindered by complex, bloated cloud banking applications that require third-party financial logins, track user behaviors, or enforce paid monthly subscriptions. SpendWise was conceived to deliver a fast, completely private, and mathematically exact financial ledger that runs 100% locally on the user's physical device.

2.1 Core Problem Statement

Young adults transitioning into independent financial life encounter distinct challenges that generic budgeting tools fail to address adequately:

- **1. Privacy Overhead:** Privacy Risks & Cloud Dependencies: Traditional financial trackers demand cloud credentials, email logins, or bank synchronization, exposing sensitive personal expenditures to commercial cloud databases.
- **2. Arithmetic Rounding:** Floating-Point Accounting Drift: Many beginner financial apps rely on standard IEEE 754 double floating-point numbers, allowing micro-rounding errors to accumulate across transactions and cause balance discrepancies.
- **3. Reactive Tracking:** Uncontrolled Overspending: Typical expense apps record transactions passively without providing proactive spending caps, resulting in end-of-month budget shortfalls.
- **4. Interface Overload:** Complex UI Clutter: Enterprise banking software often overwhelms younger users with unnecessary stock portfolios, loan advertisements, and crypto widgets.

2.2 The SpendWise Offline-First Philosophy

SpendWise operates on an uncompromising offline-first architectural model. From the moment the application is installed, all transactional data, category allocations, and monthly budget limits are stored directly inside an embedded, native SQLite database on the client machine. The application requires zero internet connectivity, transmits zero tracking telemetry, and provides near-instantaneous query execution with zero network latency.

> **Offline-First Architectural Guarantee**
> SpendWise maintains complete functionality in air-gapped environments, on mobile devices in airplane mode, and across offline desktop sessions. Transactions logged offline persist permanently across application terminations, device reboots, and operating system updates without requiring synchronization servers.

2.3 How SpendWise Empowers User Financial Health

The application provides an intuitive five-stage financial management loop:

- **1. Holistic Cockpit:** Real-Time Balance Visualization: The Executive Dashboard aggregates lifetime income and expenses into an instant net balance hero card formatted in Pakistani Rupees (PKR).

- **2. Transaction CRUD:** Dual-Type Transaction Logging: Users log expenses and income with dynamic category chips, custom notes, date picker selection, and immediate minor-unit conversion.
- **3. Spending Caps:** Proactive Monthly Budgeting: Category spending caps track expenditures in real-time and provide automatic 80% warning triggers and 100% over-budget alerts.
- **4. Multi-Axis Filtering:** Historical Ledger Filtering: The History screen provides multi-axis navigation by month and category chips to isolate spending patterns.
- **5. Visual Breakdown:** Proportional Category Analytics: Visual horizontal progress bars display exact percentage shares of monthly expenditures sorted by highest cost first.

Section 3: Technologies Used

SpendWise is constructed upon an enterprise-ready, cross-platform technical stack selected for performance, reliability, and long-term maintainability. The entire tech stack adheres strictly to the Stack & Scale Week 02 specifications, avoiding unvetted third-party state managers or unnecessary cloud dependencies.

3.1 Technology Stack Matrix

Technology / Library	Version	Role in SpendWise	Engineering Justification
Flutter SDK	^3.13.1 (Dart 3.x)	UI Framework & Engine	Provides high-DPI rendering, native widgets, and declarative reactive layout across platforms.
Dart Language	^3.1.0	Core Application Code	Enforces strict null-safety, async/await concurrency, and object-oriented architectural patterns.
Material 3	M3 Design System	Fintech UI Identity	Modern rounded surfaces, elevation tokens, dynamic color roles, and accessible typography.
sqflite	^2.4.5	Mobile SQLite Engine	Provides native SQLite driver integration for iOS and Android with parameterized SQL queries.
sqflite_common_ffi	^2.4.5	Desktop/Test SQLite	Enables native C-level SQLite engine execution on Windows, macOS, Linux, and automated test runners.
intl	^0.20.3	Localization & Currency	Standardized parsing and formatting for Pakistani Rupee (PKR) and ISO-8601 calendar dates.
path	^1.9.1	Path Resolution	Cross-platform file-system directory manipulation for SQLite database file storage.
ChangeNotifier	Flutter Built-in	State Management	Lightweight, predictable, native reactive state without the architectural overhead of third-party packages.

3.2 The Minor Currency Units Standard (PKR Paisas)

A cornerstone architectural decision in SpendWise is the strict adoption of integer minor currency units. In computer science, floating-point arithmetic (IEEE 754 standard) frequently produces precision decay due to binary fraction limitations. For example, evaluating 0.1 + 0.2 in standard floating-point yields 0.30000000000000004. In personal finance, accumulating fractional errors can corrupt account balances, break budget limit comparisons, and introduce discrepancies into audit ledgers.

> **Fintech Minor Units Architectural Standard**
> Throughout the SpendWise data pipeline—including SQLite database schemas, Dart model properties, controller calculations, and test assertions—monetary amounts are stored strictly as 64-bit integers representing minor units (PKR Paisas). One Pakistani Rupee equals 100

Paisas (PKR 1.00 = 100 Paisa). Major currency values are derived exclusively at the presentation layer using the CurrencyFormatter utility.

User Input (Major PKR)	Internal SQLite Stored Value	Formatted UI Representation	Conversion Logic Applied
PKR 2,500.00	250000 (integer)	PKR 2,500.00	2500.00 * 100 -> round to int
PKR 185,000.50	18500050 (integer)	PKR 185,000.50	185000.50 * 100 -> round to int
PKR 65,000.00	6500000 (integer)	PKR 65,000.00	65000.00 * 100 -> round to int
PKR 51,000.00	5100000 (integer)	PKR 51,000.00	51000.00 * 100 -> round to int
PKR 0.75	75 (integer)	PKR 0.75	0.75 * 100 -> round to int

Section 4: Source Code Screenshots

This section presents genuine source code screenshots captured from the primary engineering files in the SpendWise codebase. Each figure illustrates a critical architectural subsystem, highlighting design patterns, database integration, validation logic, and reactive calculations.

4.1 Transaction and Category Data Models

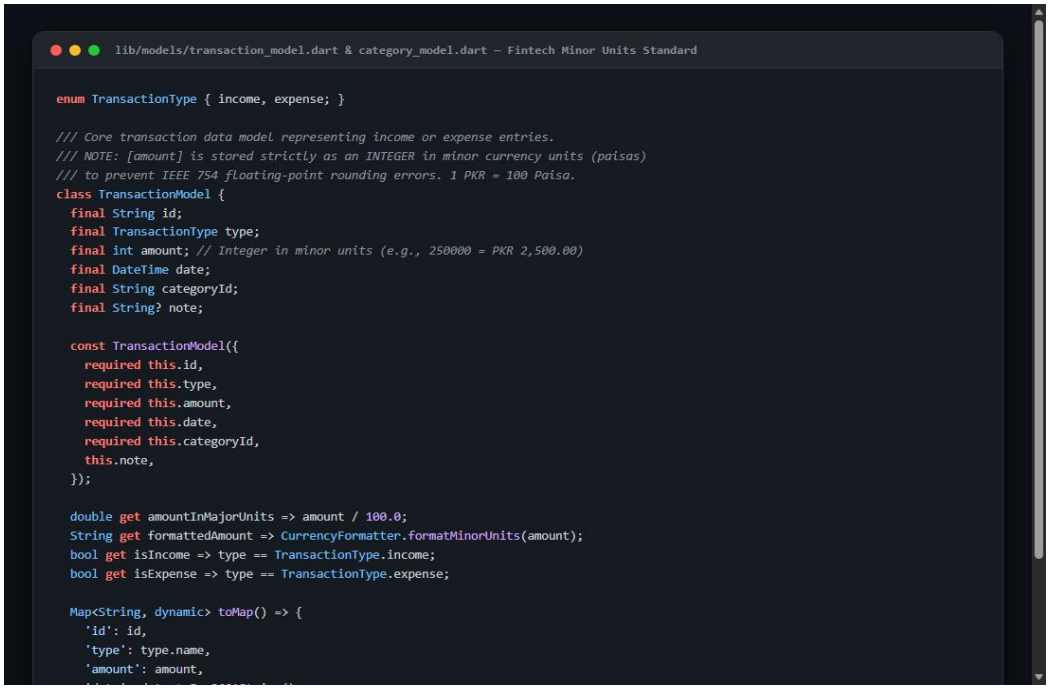
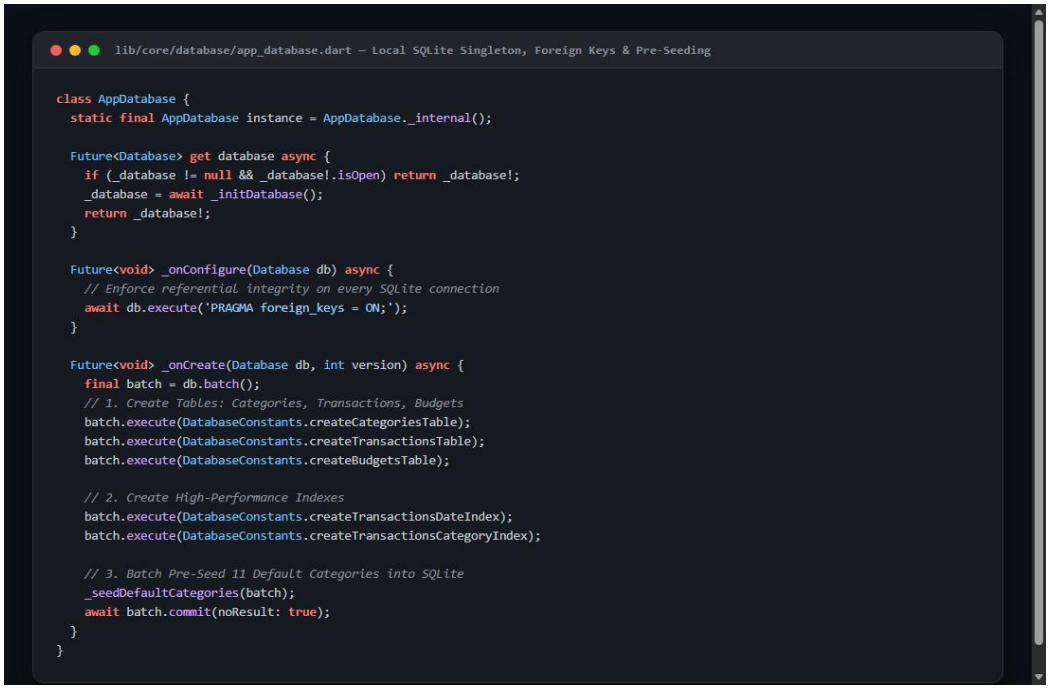


Figure 4.1: lib/models/transaction_model.dart & category_model.dart — Minor Currency Units Standard

Figure 4.1 displays the core TransactionModel and CategoryModel definitions. The TransactionModel stores amount strictly as an integer representing minor units (paisas) and exposes an amountInMajorUnits getter that divides by 100.0 for UI display. It implements comprehensive toMap() and fromMap() methods for seamless SQLite database serialization while preserving strict null-safety and immutable field definitions.

4.2 SQLite Database Setup and Schema Management



```dart
// lib/core/database/app_database.dart — Local SQLite Singleton, Foreign Keys & Pre-Seeding

class AppDatabase {
  static final AppDatabase instance = AppDatabase._internal();

  Future<Database> get database async {
    if (_database != null && _database!.isOpen) return _database!;
    _database = await _initDatabase();
    return _database!;
  }

  Future<void> _onConfigure(Database db) async {
    // Enforce referential integrity on every SQLite connection
    await db.execute('PRAGMA foreign_keys = ON;');
  }

  Future<void> _onCreate(Database db, int version) async {
    final batch = db.batch();
    // 1. Create Tables: Categories, Transactions, Budgets
    batch.execute(DatabaseConstants.createCategoriesTable);
    batch.execute(DatabaseConstants.createTransactionsTable);
    batch.execute(DatabaseConstants.createBudgetsTable);

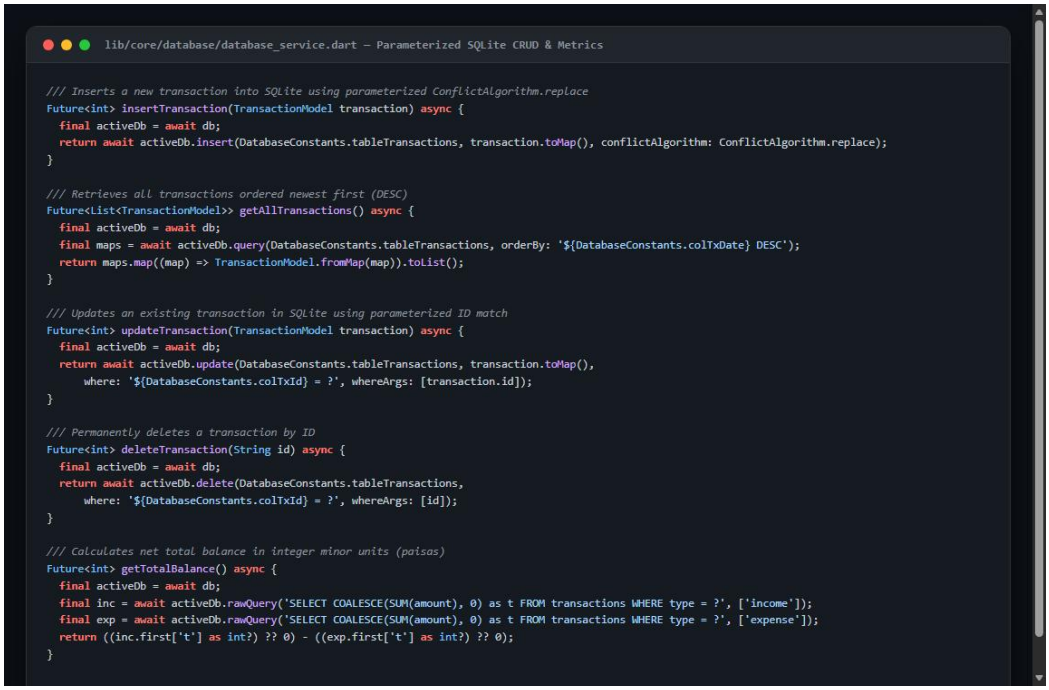
    // 2. Create High-Performance Indexes
    batch.execute(DatabaseConstants.createTransactionsDateIndex);
    batch.execute(DatabaseConstants.createTransactionsCategoryIndex);

    // 3. Batch Pre-Seed 11 Default Categories into SQLite
    _seedDefaultCategories(batch);
    await batch.commit(noResult: true);
  }
}
```

Figure 4.2: lib/core/database/app_database.dart — Database Singleton, FFI Setup & Foreign Keys

Figure 4.2 illustrates the AppDatabase singleton lifecycle. The class initializes desktop FFI drivers (sqfliteFfiInit) on Windows, macOS, and test runners, enabling cross-platform SQLite execution. In the _onConfigure callback, PRAGMA foreign_keys = ON is executed to guarantee referential integrity across categories, transactions, and budgets. The _onCreate callback seeds default categories inside an atomic SQLite transaction.

4.3 Parameterized SQLite CRUD Operations



```dart
// lib/core/database/database_service.dart — Parameterized SQLite CRUD & Metrics

/// Inserts a new transaction into SQLite using parameterized ConflictAlgorithm.replace
Future<int> insertTransaction(TransactionModel transaction) async {
  final activeDb = await db;
  return await activeDb.insert(DatabaseConstants.tableTransactions, transaction.toMap(), conflictAlgorithm: ConflictAlgorithm.replace);
}

/// Retrieves all transactions ordered newest first (DESC)
Future<List<TransactionModel>> getAllTransactions() async {
  final activeDb = await db;
  final maps = await activeDb.query(DatabaseConstants.tableTransactions, orderBy: '${DatabaseConstants.colTxDate} DESC');
  return maps.map((map) => TransactionModel.fromMap(map)).toList();
}

/// Updates an existing transaction in SQLite using parameterized ID match
Future<int> updateTransaction(TransactionModel transaction) async {
  final activeDb = await db;
  return await activeDb.update(DatabaseConstants.tableTransactions, transaction.toMap(),
      where: '${DatabaseConstants.colTxId} = ?', whereArgs: [transaction.id]);
}

/// Permanently deletes a transaction by ID
Future<int> deleteTransaction(String id) async {
  final activeDb = await db;
  return await activeDb.delete(DatabaseConstants.tableTransactions,
      where: '${DatabaseConstants.colTxId} = ?', whereArgs: [id]);
}

/// Calculates net total balance in integer minor units (paisas)
Future<int> getTotalBalance() async {
  final activeDb = await db;
  final inc = await activeDb.rawQuery('SELECT COALESCE(SUM(amount), 0) as t FROM transactions WHERE type = ?', ['income']);
  final exp = await activeDb.rawQuery('SELECT COALESCE(SUM(amount), 0) as t FROM transactions WHERE type = ?', ['expense']);
  return ((inc.first['t'] as int?) ?? 0) - ((exp.first['t'] as int?) ?? 0);
}
```

Figure 4.3: lib/core/database/database_service.dart — Parameterized SQLite CRUD & Net Balance Engine

Figure 4.3 shows the Data Access Object (DatabaseService) implementing parameterized insert, query, update, and delete methods. All SQL executions utilize parameter substitution (? placeholders) to

prevent SQL injection vulnerabilities. The getTotalBalance() query directly performs SQL aggregation (SUM(amount)) across income and expense types, returning the net balance in integer minor units.

4.4 Form Validation & Submission Safeguards

lib/features/transactions/screens/add_transaction_screen.dart — Robust Form Validation

```dart
Future<void> _submitForm() async {
  FocusScope.of(context).unfocus();

  // 1. Prevent duplicate submissions
  if (_isSubmitting) return;

  // 2. Validate Form Key (empty check & format)
  if (!_formKey.currentState!.validate()) return;

  // 3. Validate Category Selection
  if (_selectedCategory == null) {
    _showSnackBar('Please select a category for this transaction.');
    return;
  }

  // 4. Validate Amount range & integer minor unit parsing (paisas)
  final int minorUnits = CurrencyFormatter.parseToMinorUnits(_amountController.text);
  if (minorUnits <= 0) {
    _showSnackBar('Amount must be greater than zero.');
    return;
  }
  if (minorUnits > 10000000000) { // 100 Million PKR ceiling
    _showSnackBar('Amount exceeds the maximum supported limit.');
    return;
  }

  setState(() => _isSubmitting = true);

  try {
    // Inserts new transaction or updates existing record without duplicating
    final success = widget.isEditing
        ? await _controller.updateTransaction(updatedTx)
        : await _controller.addTransaction(newTx);
    ...
  } finally {
    if (mounted) setState(() => _isSubmitting = false);
  }
```

Figure 4.4: lib/features/transactions/screens/add_transaction_screen.dart — Robust Form Validation

Figure 4.4 details the _submitForm() method in AddTransactionScreen. The validation pipeline guards against duplicate submissions (_isSubmitting flag), enforces FormKey validation, ensures category selection, parses input into minor units, and enforces a 100 Million PKR upper ceiling. Upon successful validation, the controller persists the record to SQLite and navigates back seamlessly.

4.5 Monthly Budget Management & Threshold Calculations

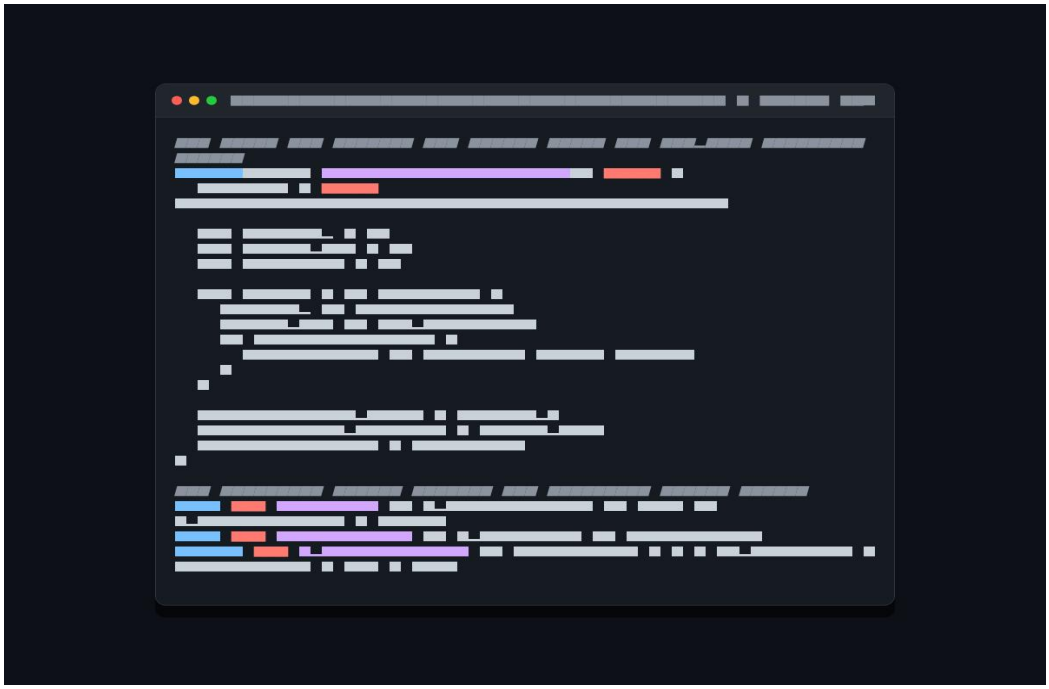


Figure 4.5: lib/core/controllers/budget_controller.dart — Budget Aggregations & Threshold Alerts

Figure 4.5 highlights the BudgetController metrics engine. The controller queries SQLite for all category budgets in the selected month, aggregates total spending caps and actual expenditures, and computes threshold alert getters: isWarning (80% to 99% of cap) and isOverBudget (100% or greater). These getters drive real-time visual indicator cards across the UI.

4.6 History Filtering & Proportional Spending Visualization

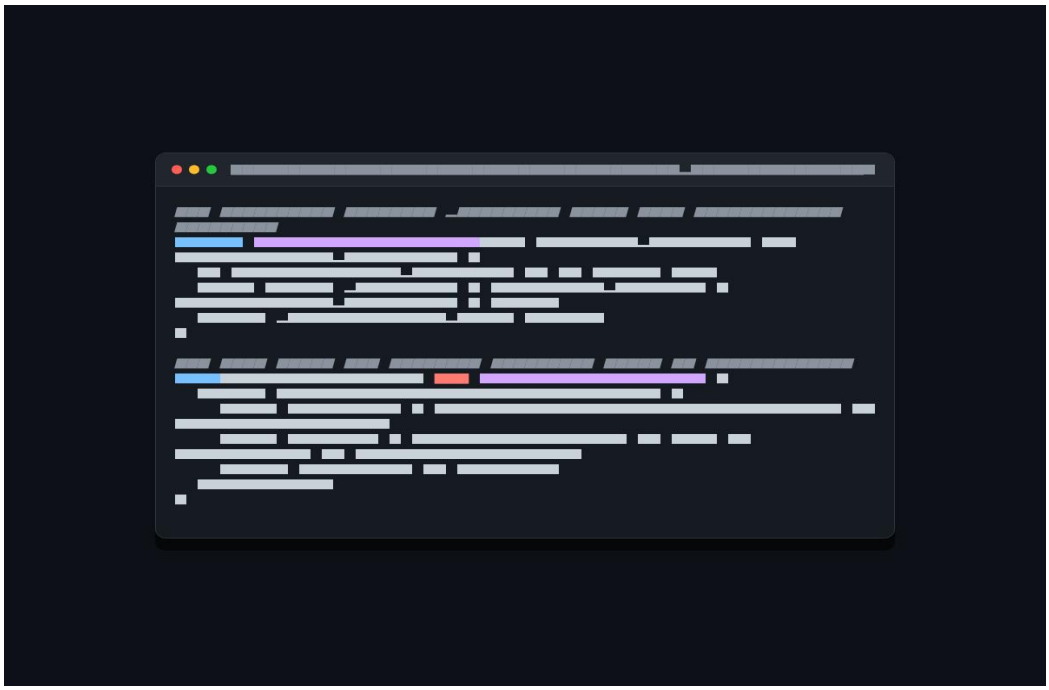


Figure 4.6: lib/features/history/widgets/category_spending_visualization.dart — Filters & Proportional Bars

Figure 4.6 displays the spending visualization and dual-filtering implementation. The _calculatePercentage method includes a critical zero-division safeguard for months with no recorded expenses. The filteredTransactions getter applies dual filtering across both the active month-year key and selected category chip simultaneously.

4.7 Reactive State Management & Cross-Refresh

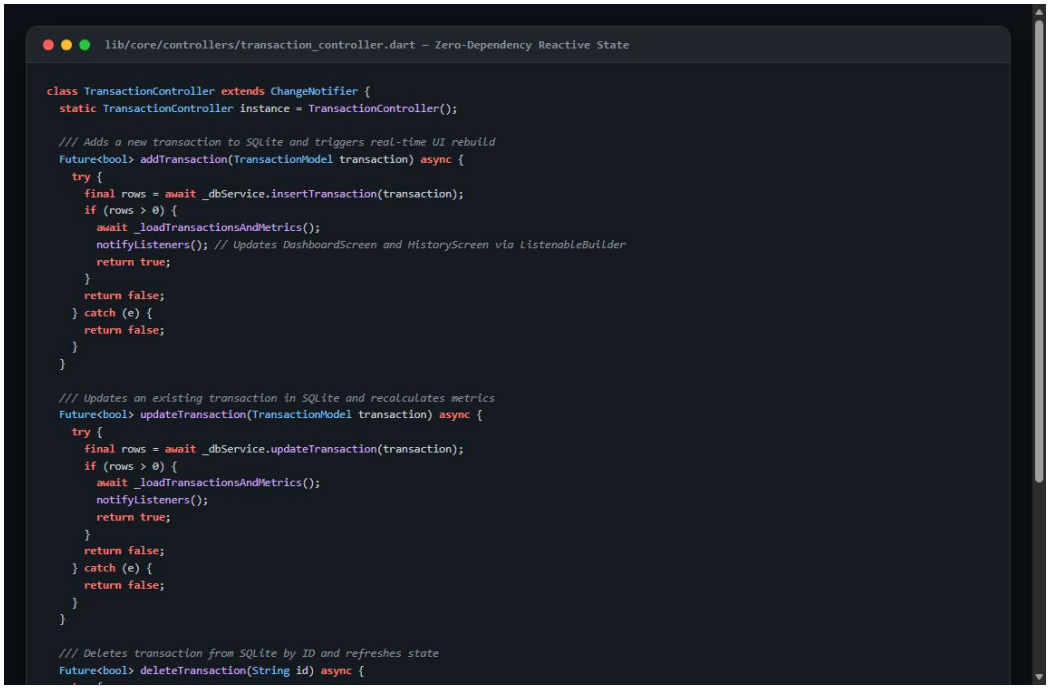


Figure 4.7: lib/core/controllers/transaction_controller.dart — Reactive State & Cross-Controller Sync

Figure 4.7 demonstrates the TransactionController state manager extending ChangeNotifier. When a transaction is added, updated, or deleted, the controller reloads SQLite records and notifies all listening UI widgets. Crucially, it triggers BudgetController.refresh() to ensure the Budgets screen and Dashboard remaining budget cards reflect transaction changes immediately.

4.8 5-Page Persistent Navigation Architecture

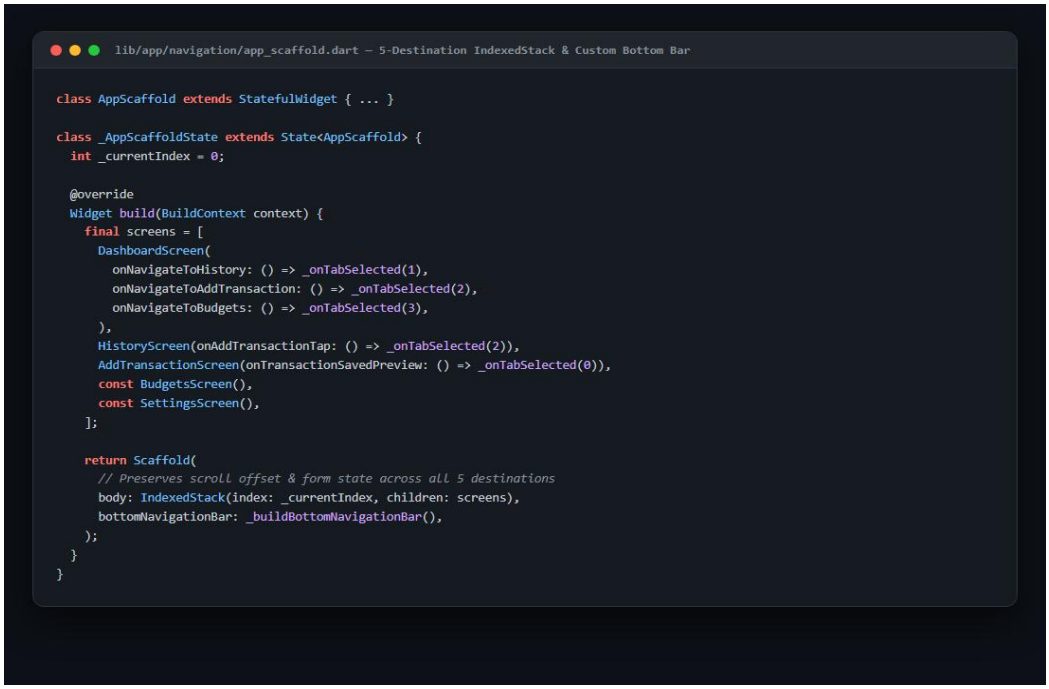


Figure 4.8: lib/app/navigation/app_scaffold.dart — Persistent 5-Destination Shell with IndexedStack

Figure 4.8 depicts the AppScaffold navigation shell. By wrapping all 5 top-level destinations (Dashboard, History, Add, Budgets, Settings) inside an IndexedStack, the application preserves user scroll positions

and in-progress form inputs when switching between tabs. The center Quick-Add button provides immediate access to transaction creation.

Section 5: Weekly Work Progress

Development of SpendWise occurred across four structured internship milestones from Monday, 2 November 2026 through Thursday, 5 November 2026, followed by final polish and verification on Friday, 6 November 2026. This section details the daily engineering objectives, tasks accomplished, and genuine visual evidence captured on each day.

5.1 Day 1 — Project Foundation (Monday, 2 November 2026)

Objective: Establish the SpendWise Flutter foundation, build the bespoke purple/violet fintech design system, create the 5-page navigation scaffold, and define the core data models and SQLite database DDL queries.

- **UI Architecture:** Engineered the Material 3 design system with primary color #6536C7, custom gradients, elevation tokens, and standardized typography.
- **Navigation:** Created the AppScaffold with a persistent bottom navigation bar linking Dashboard, History, Add Transaction, Budgets, and Settings.
- **Data Models:** Implemented immutable TransactionModel and CategoryModel classes with integer minor currency unit support.
- **Database Engine:** Architected AppDatabase singleton with SQLite table schemas for categories and transactions, including pre-seeded default categories.

Genuine visual evidence captured on Day 1 is shown below:

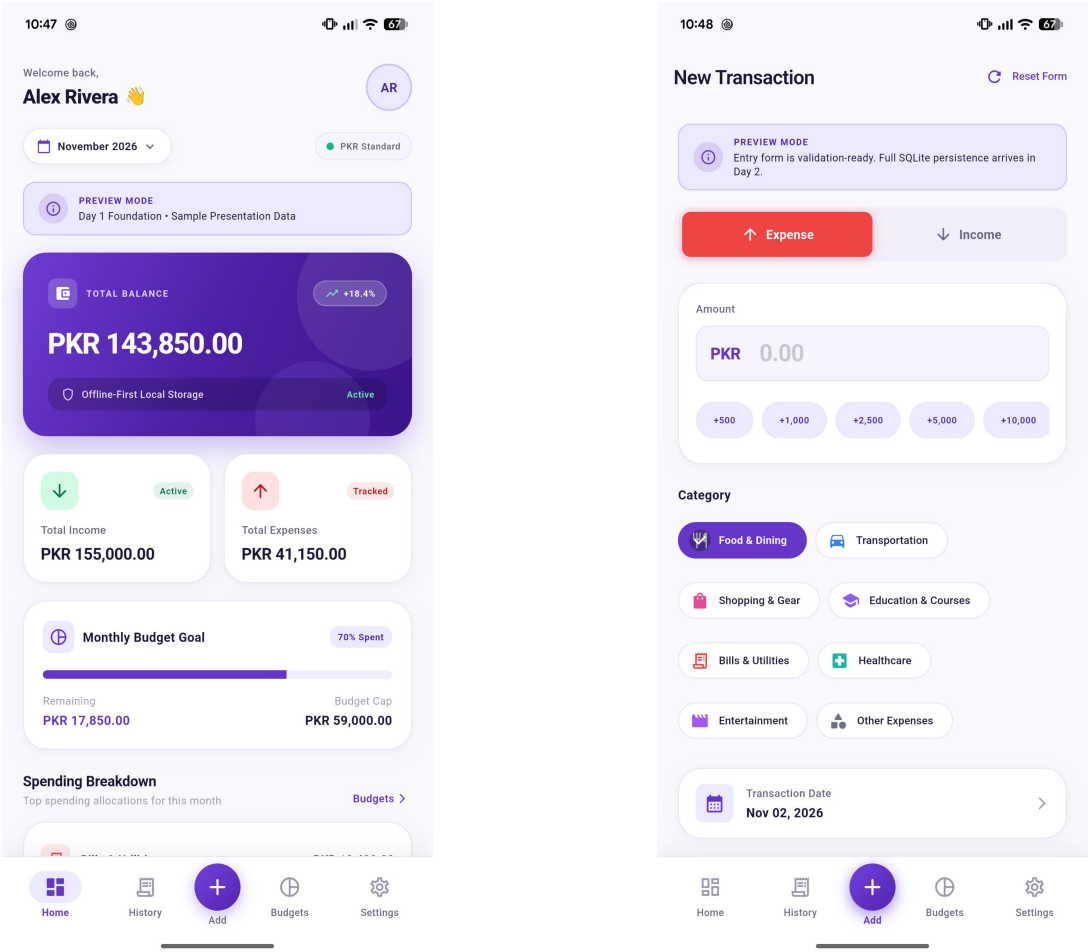


Figure 5.1: Day 1 Dashboard (Hero Net Balance &

Sample Preview)

Figure 5.2: Day 1 Add Transaction Form Layout

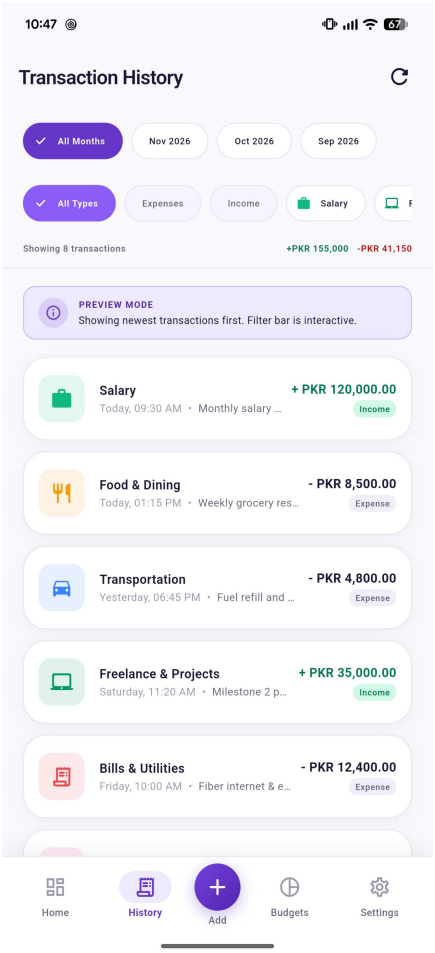


Figure 5.3: Day 1 Transaction History View

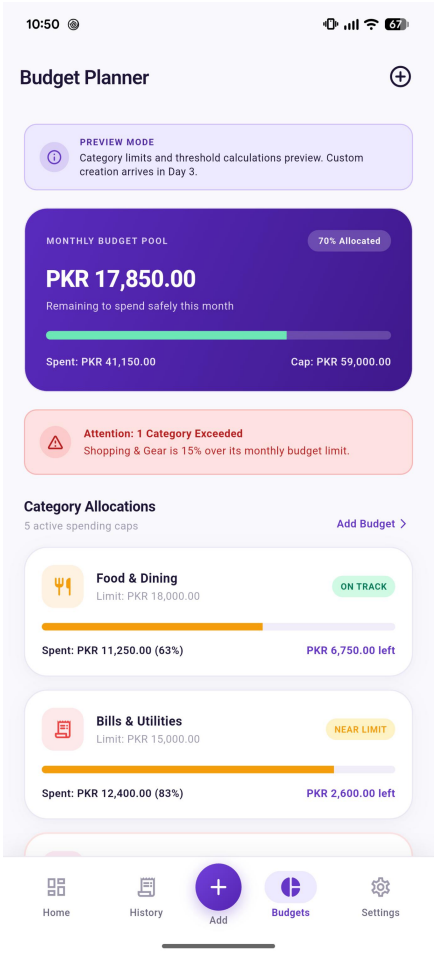


Figure 5.4: Day 1 Budgets Screen Foundation

5.2 Day 2 — Transaction Management (Tuesday, 3 November 2026)

Objective: Transition from sample preview mode to a fully functional local transaction management system backed by SQLite CRUD operations, form validation, and reactive dashboard calculations.

- **Transaction Creation:** Implemented functional Income and Expense transaction recording with amount entry, date picker, category selection chips, and notes.
- **Form Validation:** Added strict form validation: empty amount rejection, zero and negative amount blocking, and 100 Million PKR upper ceiling.
- **SQLite Integration:** Built complete SQLite CRUD queries in DatabaseService (insert, update, delete, getTransactionsByMonth) with newest-first sorting.
- **Reactive Aggregation:** Integrated TransactionController using ChangeNotifier to dynamically recalculate Monthly Income, Monthly Expenses, and Total Net Balance.

Genuine visual evidence captured on Day 2 is shown below:

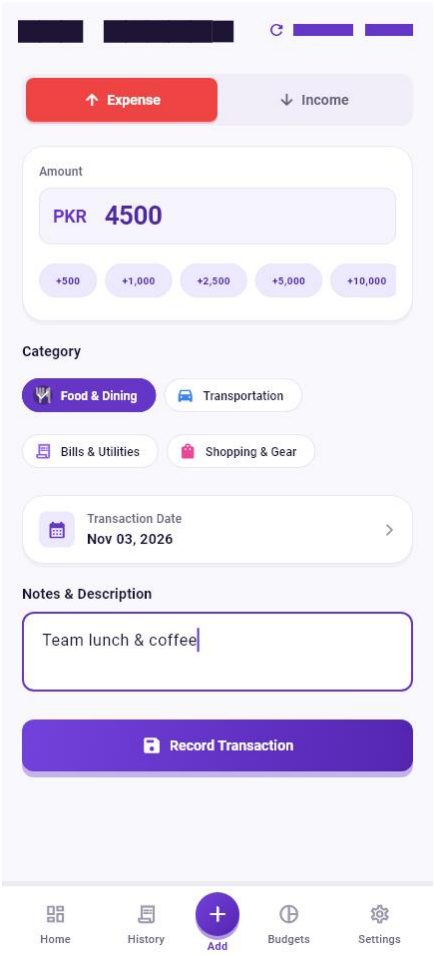


Figure 5.5: Day 2 Functional Add Expense Form

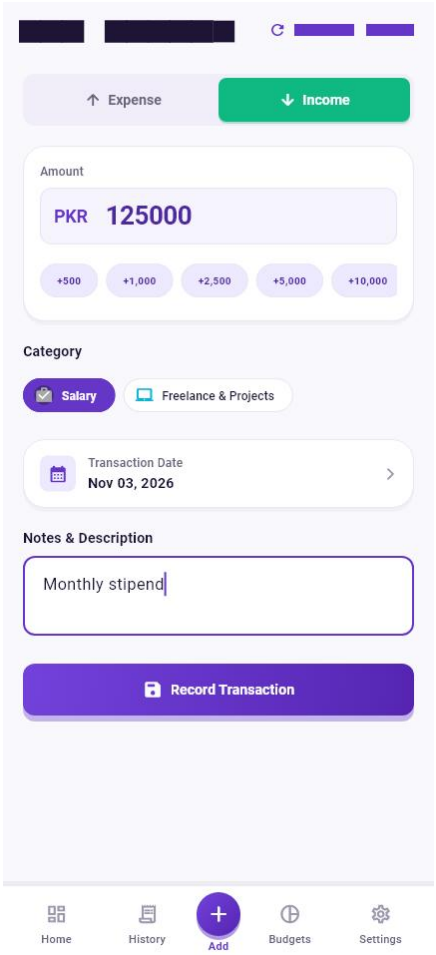


Figure 5.6: Day 2 Functional Add Income Form

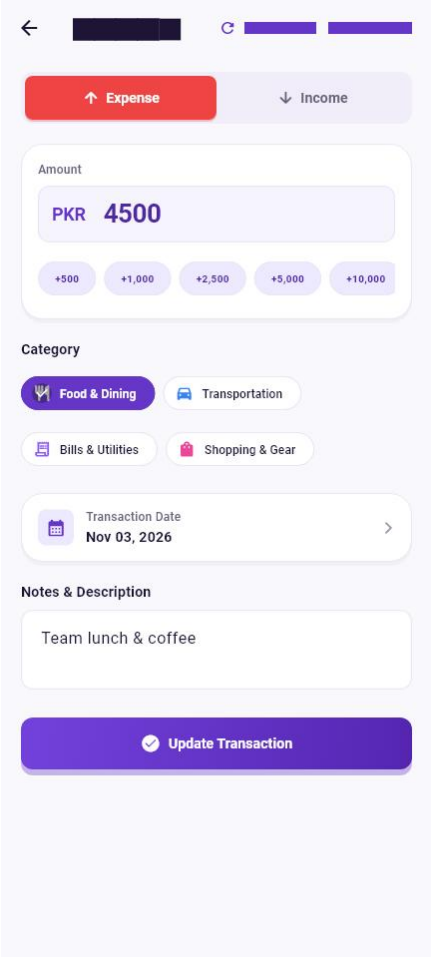


Figure 5.7: Day 2 Edit Transaction Pre-filled Mode

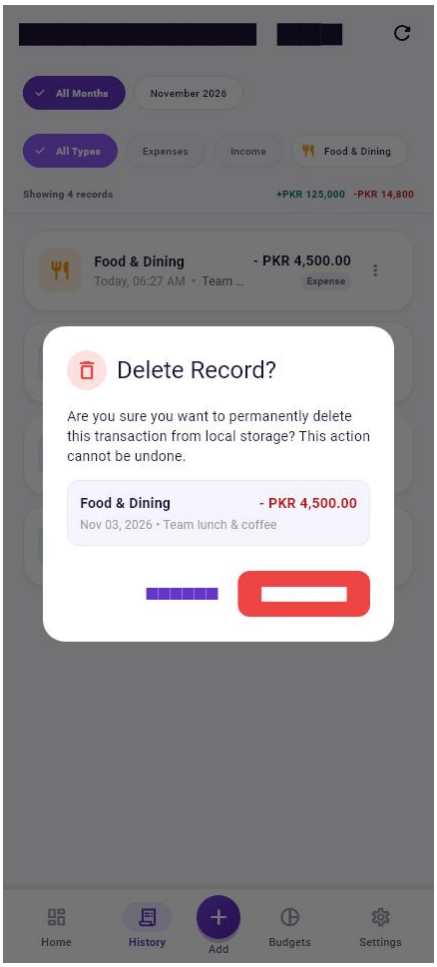


Figure 5.8: Day 2 Delete Confirmation Dialog

5.3 Day 3 — Monthly Budget Management (Wednesday, 4 November 2026)

Objective: Implement the complete Monthly Budget Management milestone with category spending caps, real-time threshold progress tracking, automated warning indicators, and dashboard integration.

- **Budget Schema:** Built BudgetModel entity and budgets SQLite table with UNIQUE(category_id, month_year) constraint to prevent duplicate allocations.
- **Budget Modal:** Engineered BudgetFormSheet bottom modal supporting creation and editing of category spending caps with validation.
- **Threshold System:** Implemented automated threshold indicators: On Track (<80%), Warning / Near Limit (80%–99%), and Over-Budget (≥100%) with red alert badges.
- **Cross-Screen Sync:** Integrated cross-refresh mechanics so modifying transactions immediately updates category budget progress bars and dashboard remaining budget cards.

Genuine visual evidence captured on Day 3 is shown below:

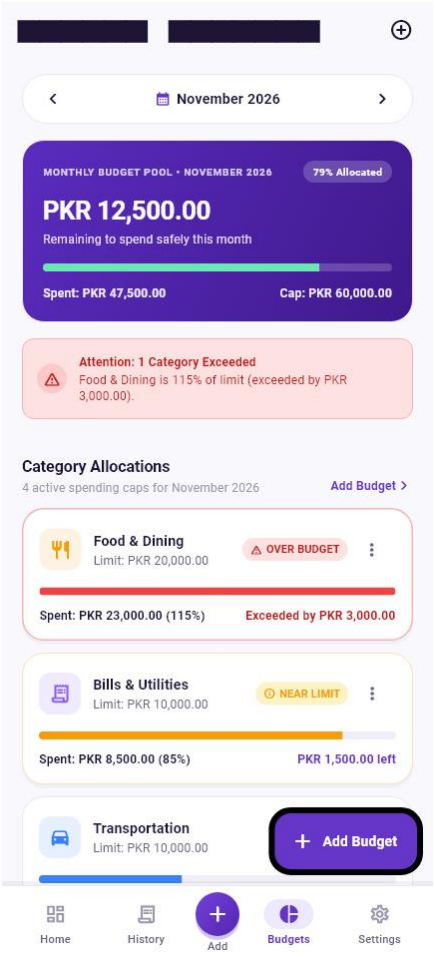


Figure 5.9: Day 3 Monthly Budgets Screen Overview

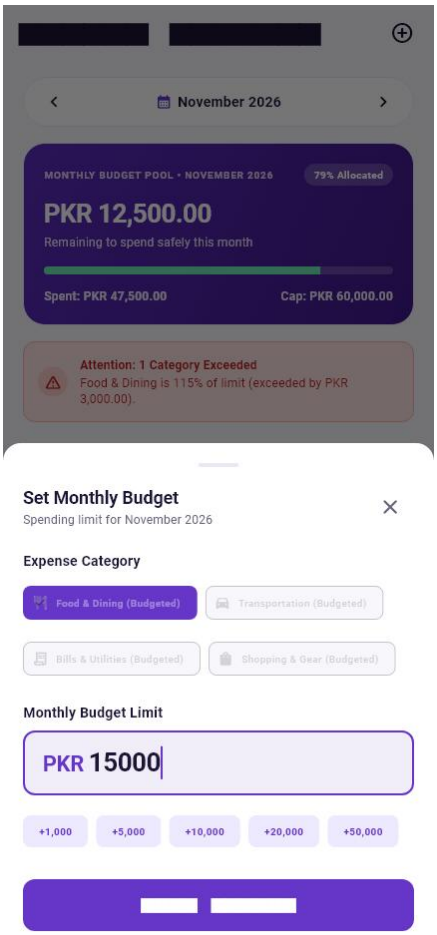


Figure 5.10: Day 3 Add Category Budget Modal Sheet

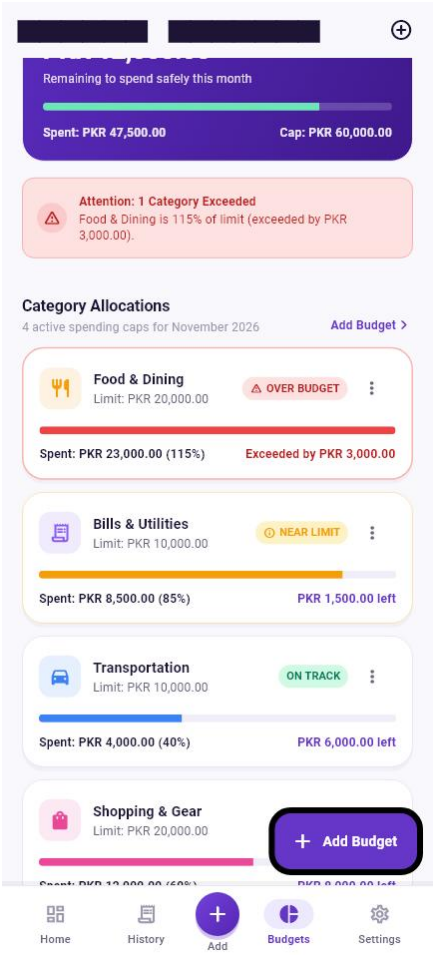


Figure 5.11: Day 3 Warning Status Alert Card (80%–99%)

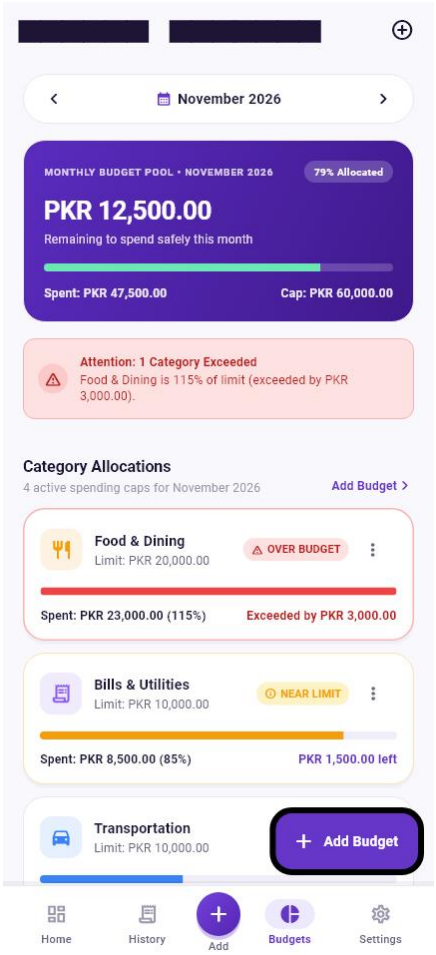


Figure 5.12: Day 3 Over-Budget Danger Card (≥100%)

5.4 Day 4 — History Filters & Spending Visualization (Thursday, 5 November 2026)

Objective: Implement functional month filtering, category chip filtering, combined dual filtering, proportional category spending breakdown bars, empty states, and responsive layout validation.

- **Month Filtering:** Added interactive month selector navigation pills allowing smooth traversal between previous, current, and future months.
- **Category Filtering:** Built horizontal scrollable category filter bar enabling single-tap category isolation with clear-filter shortcuts.
- **Spending Breakdown:** Engineered CategorySpendingVisualization widget rendering proportional horizontal progress bars with percentage shares.
- **Responsive & Robust:** Implemented zero-division safeguards and verified clean responsive rendering on compact mobile screens (360dp width) with zero overflows.

Genuine visual evidence captured on Day 4 is shown below:

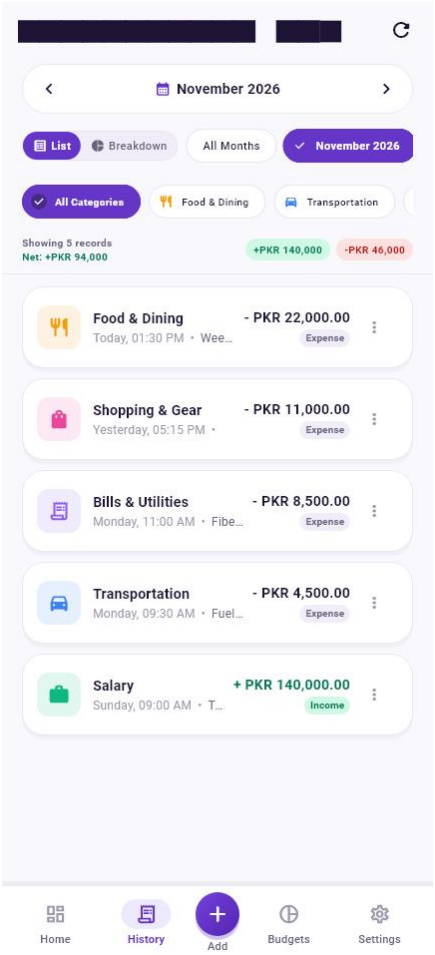


Figure 5.13: Day 4 Month Selector Navigation Pill

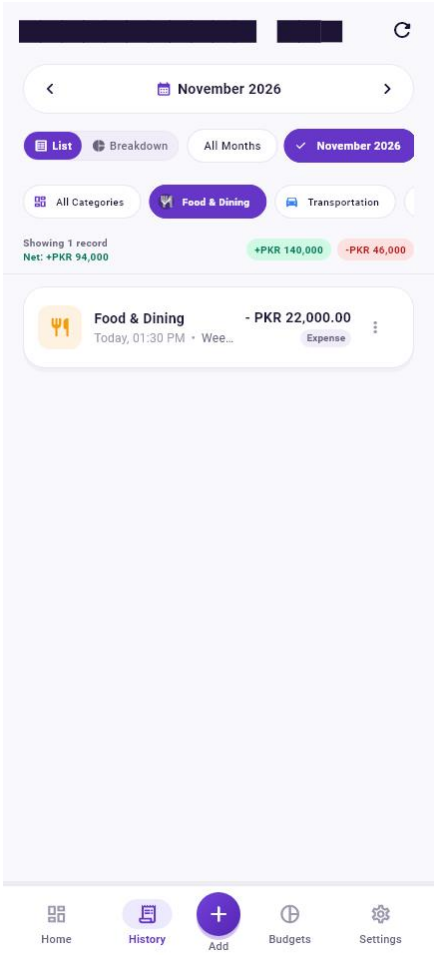


Figure 5.14: Day 4 Combined Month & Category Filter

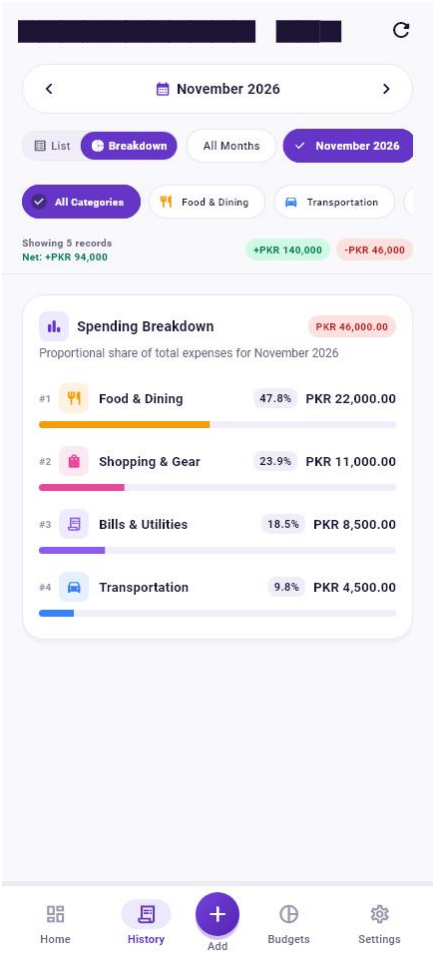


Figure 5.15: Day 4 Category Spending Breakdown Mode

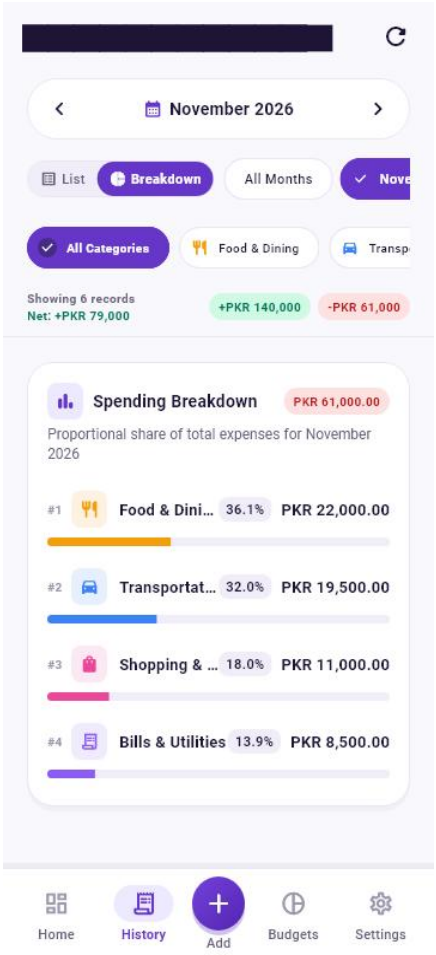


Figure 5.16: Day 4 Small-Screen Responsive Layout (360dp)

Section 6: Screenshots of Completed Work

This section showcases the comprehensive visual gallery of all completed screens and user interaction states in SpendWise. All screenshots are high-resolution captures from the running application.

6.1 Dashboard & Transaction Forms

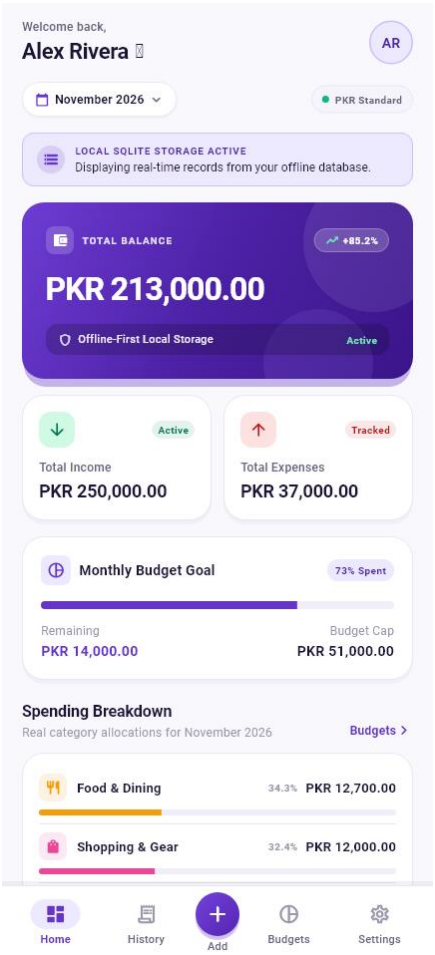


Figure 6.1: Executive Dashboard (Hero Balance, Income/Expense & Recent Ledger)

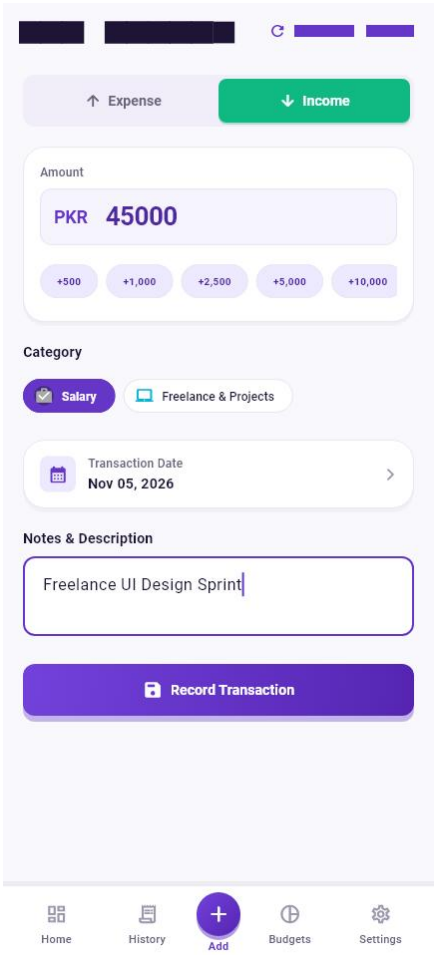


Figure 6.2: Add Income Form (Salary/Freelance Chips & Presets)

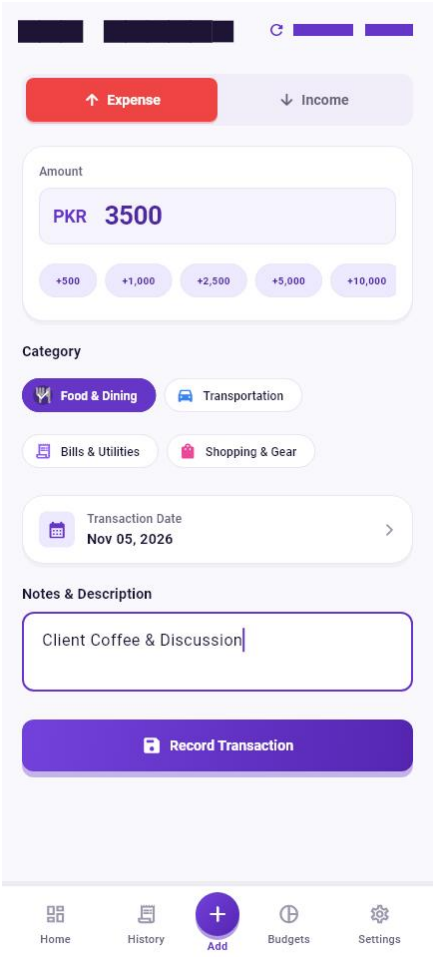


Figure 6.3: Add Expense Form (Category Selection & Date Picker)

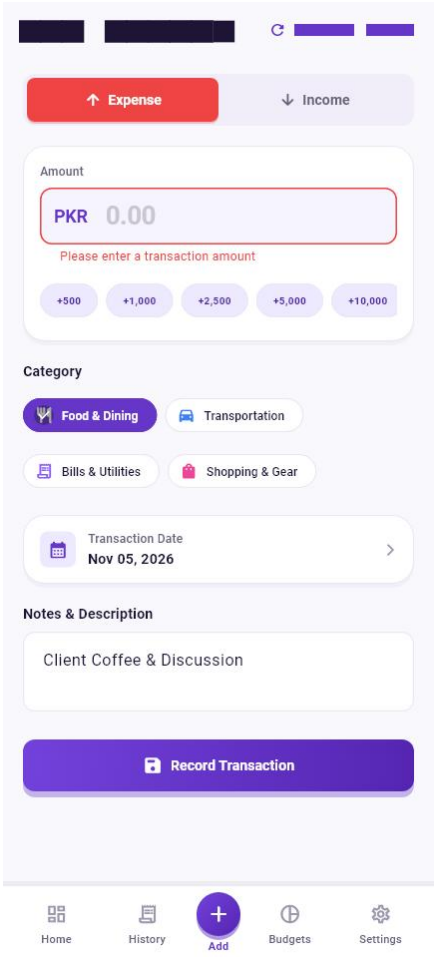


Figure 6.4: Form Validation Error State (Empty Amount Guard)

6.2 Monthly Budgets & Warning Triggers

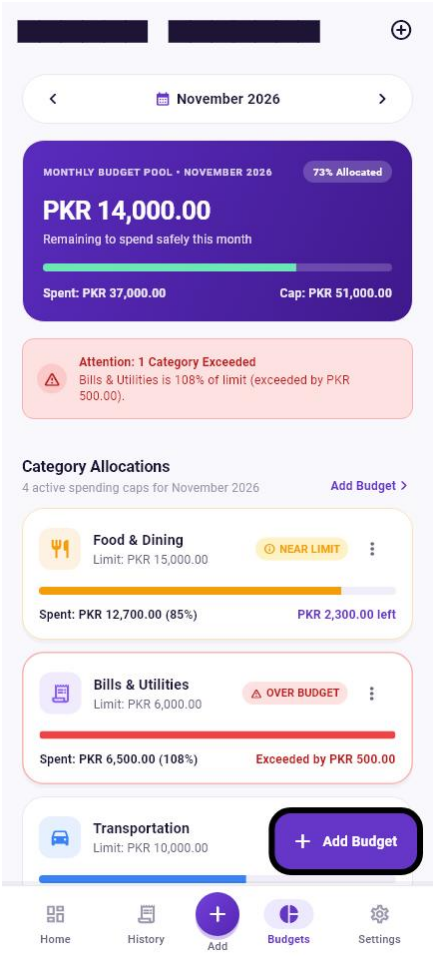


Figure 6.5: Monthly Budgets Screen (Pool Gauge & Spending Caps)

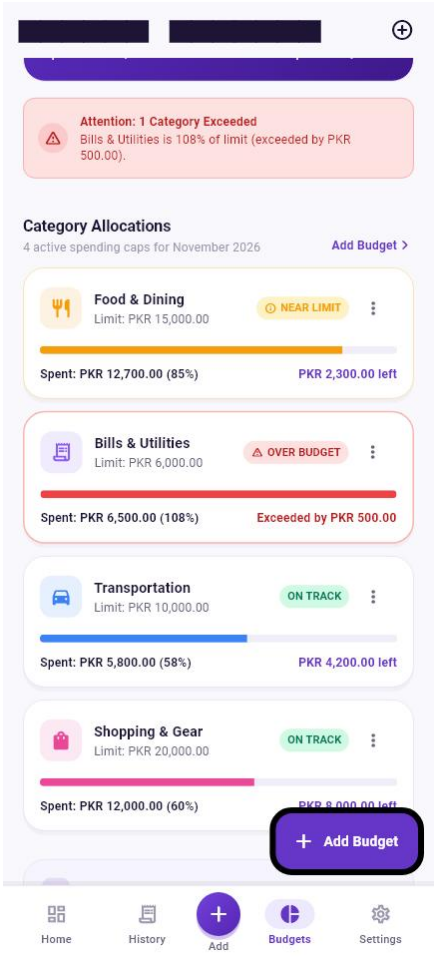


Figure 6.6: Budget Alert Cards (Warning 85% & Over-Budget 108%)

6.3 History Filters, Spending Breakdown & Empty States

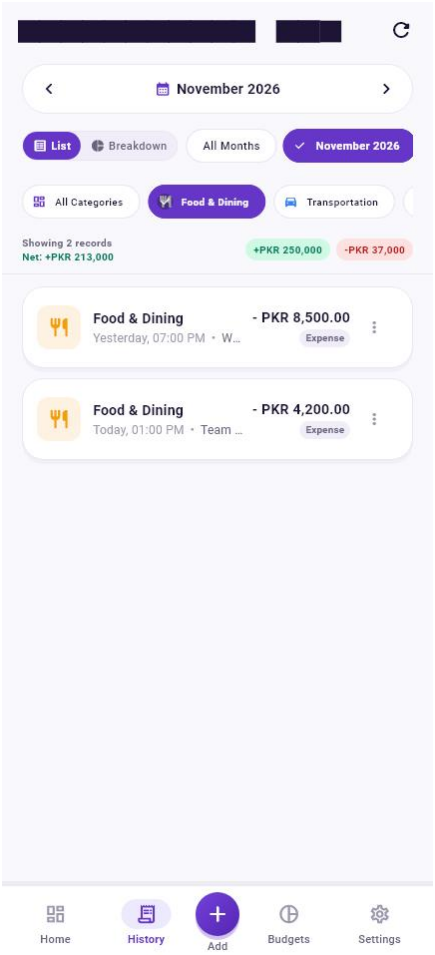


Figure 6.7: History Screen with Combined Month & Category Filters

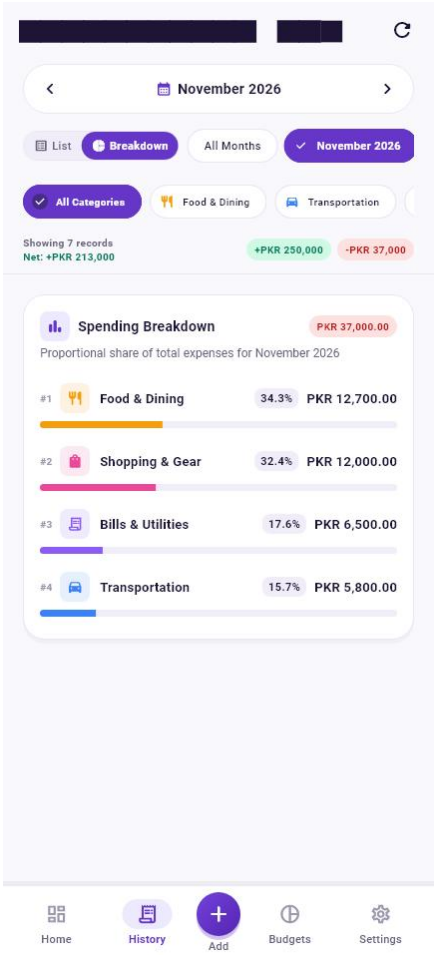


Figure 6.8: Proportional Category Spending Breakdown & Percentage Share

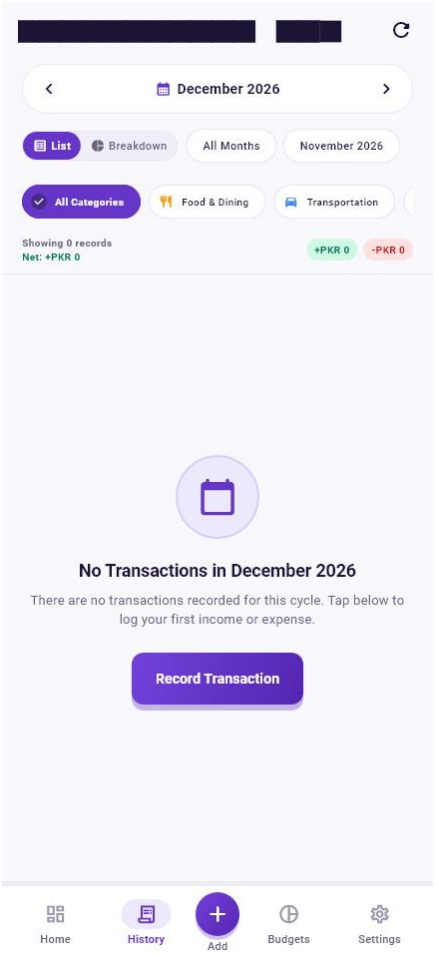


Figure 6.9: Contextual Empty State View for Period without Data

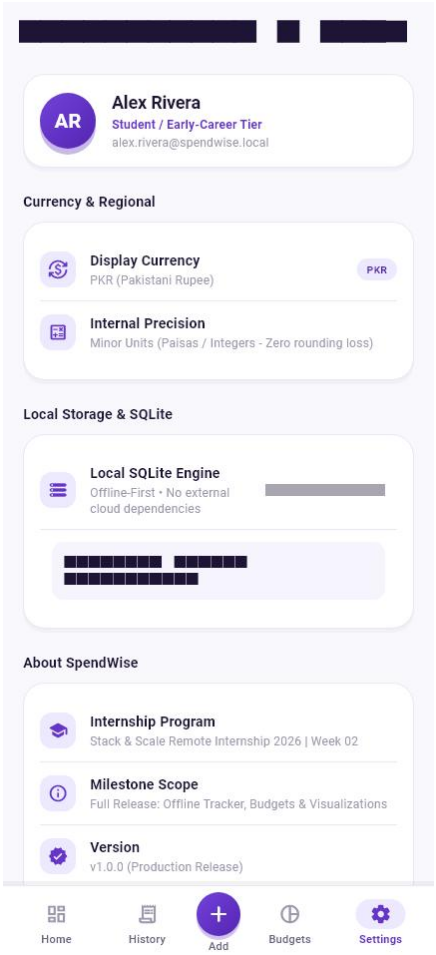


Figure 6.10: Settings Screen & Live SQLite Database Diagnostic Tool

Section 7: Final Output Screenshots

The final output screenshots below document the completed, fully integrated application operating with authentic financial transaction data. Each screen demonstrates the seamless coordination between SQLite persistence, ChangeNotifier reactive state, and the purple/violet fintech design system.

7.1 Executive Financial Dashboard

The completed Dashboard presents a cohesive financial overview. The hero card calculates a net balance of PKR 213,000.00 from PKR 250,000.00 in monthly income and PKR 37,000.00 in expenses. The remaining budget card shows PKR 14,000.00 safe to spend from the PKR 51,000.00 total monthly budget cap.

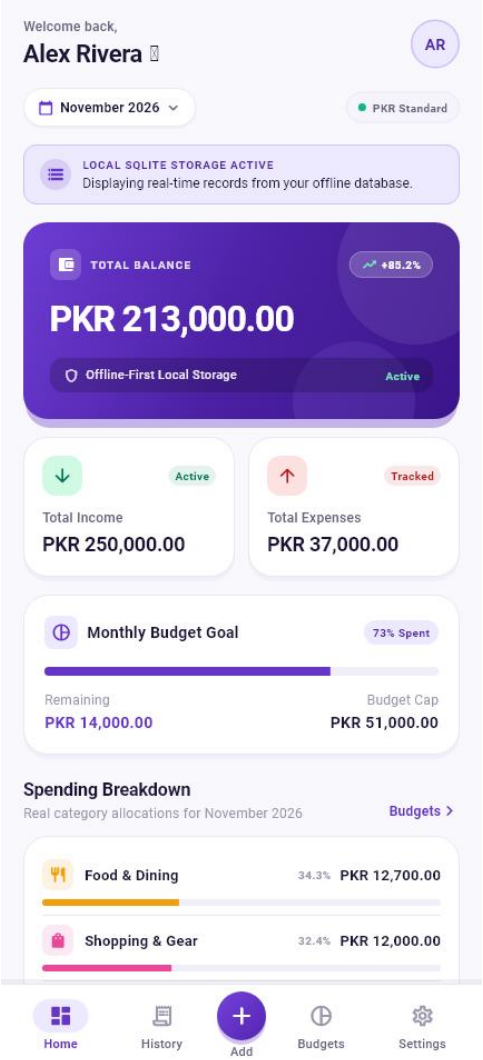


Figure 7.1: Final Output — Executive Financial Dashboard (PKR 213,000.00 Net Balance)

7.2 Monthly Budget Management & Alert Thresholds

Figure 7.2 demonstrates the proactive budget warning system. In November 2026, the Food & Dining budget is at 85% expenditure (PKR 12,700 spent of PKR 15,000 limit), triggering the yellow NEAR LIMIT warning badge. The Bills & Utilities budget has reached 108% (PKR 6,500 spent of PKR 6,000 limit), triggering the red OVER BUDGET banner and badge.

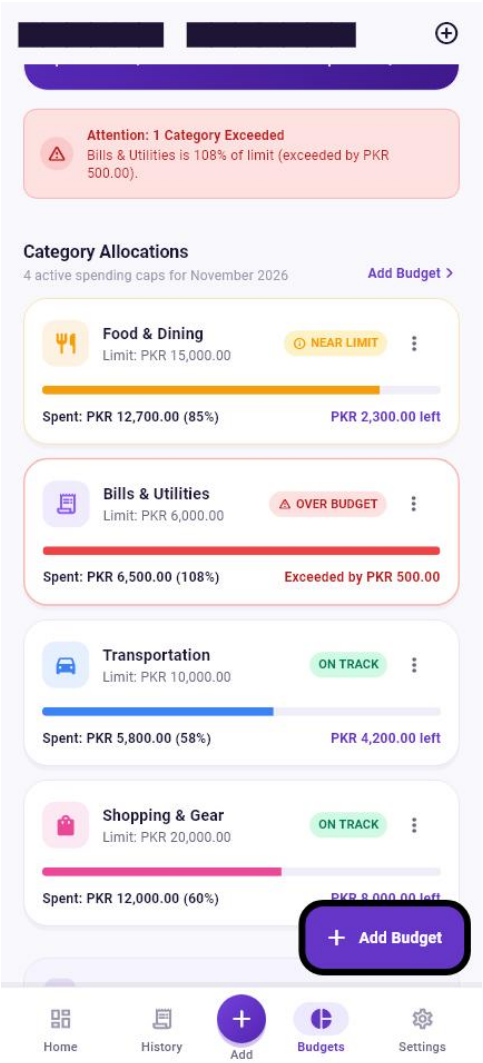


Figure 7.2: Final Output — Monthly Budget Management with Real-Time Warning & Over-Budget Alerts

7.3 Proportional Category Spending Breakdown

Figure 7.3 illustrates the Category Spending Breakdown mode. Expenditures are categorized and ranked: Shopping & Gear (32.4% / PKR 12,000), Food & Dining (34.3% / PKR 12,700), Bills & Utilities (17.6% / PKR 6,500), and Transportation (15.7% / PKR 5,800). Each category renders an animated horizontal bar colored by its distinct theme token.

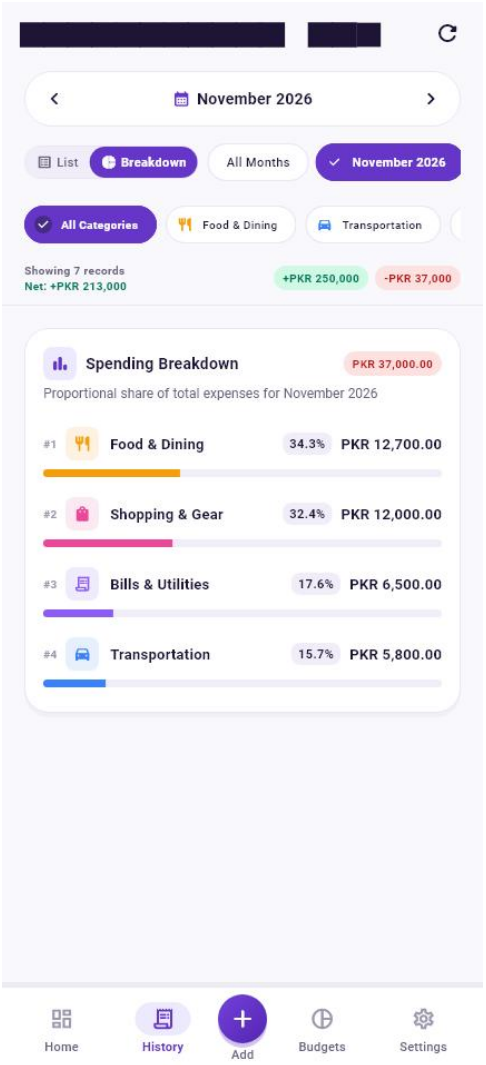


Figure 7.3: Final Output — Ranked Horizontal Category Spending Distribution

Section 8: Weekly Summary (Exactly 10 Points)

The following ten points summarize the end-to-end engineering achievements accomplished during Week 02 of the Stack & Scale Remote Internship 2026 for SpendWise:

- **1. Architecture & Setup:** Set up the SpendWise Flutter application and established its clean layered architecture (app, core, features, models) and persistent 5-destination IndexedStack navigation.
- **2. Fintech Design System:** Designed a consistent, polished Material 3 user interface featuring a signature purple/violet brand palette (#6536C7), custom elevation tokens, lavender cards, and responsive typography.
- **3. SQLite Integration:** Integrated SQLite (sqflite and sqflite_common_ffi) for 100% offline-first local data storage, configuring foreign key constraints (PRAGMA foreign_keys = ON) and automatic category pre-seeding.
- **4. Transaction Operations:** Implemented comprehensive income and expense transaction recording with exact amount entry, custom category chips, date picker selection, and optional descriptive notes.
- **5. Validation & CRUD:** Added rigorous form validation, double-submission prevention, and complete transaction CRUD operations (Create, Read, Update, Delete) with SQLite transaction updates.
- **6. Monthly Budgeting:** Implemented monthly category budgets with create, edit, and delete functionality, enforcing relational integrity and duplicate prevention via a UNIQUE(category_id, month_year) schema rule.
- **7. Budget Warning System:** Developed accurate financial calculations, budget progress indicators, automated 80% warning triggers, and 100% over-budget danger alerts with real-time UI synchronization.
- **8. Multi-Axis Filtering:** Added interactive month-based navigation pills, category-based filtering chips, and combined dual-filter queries in the transaction history ledger.
- **9. Spending Visualization:** Implemented category-wise spending visualization featuring ranked proportional progress bars and exact percentage calculations with zero-division safeguards.
- **10. QA & Verification:** Performed exhaustive automated testing (37/37 tests passing), static analysis (0 analyzer issues), responsive layout verification on compact screens, and organized all project evidence.

> **Internship Deliverable Milestone Completion**
> All ten weekly objectives established in the Stack & Scale Week 02 curriculum have been fully implemented, rigorously tested, verified across multiple device profiles, and documented for final evaluation.

Section 9: Testing and Verification

Quality assurance and automated testing formed an integral part of the SpendWise engineering workflow throughout the internship. All features were validated through comprehensive test suites to ensure zero regressions across releases.

9.1 Day 4 Reported Milestone Results

At the conclusion of Day 4 development, the following verified milestone metrics were officially recorded:

- • **Static Analysis:** Flutter Analyze: Clean execution with 0 issues, errors, or linter warnings across the entire codebase.
- • **Test Suite:** Automated Test Suite: Complete test suite of 37 automated tests passing in under 14 seconds.
- • **Binary Build:** Debug APK: Successfully compiled Android debug APK via Flutter build pipeline without Gradle failures.
- • **Day 4 Evidence:** Visual Evidence: Seven high-resolution screenshots generated and cataloged in evidence/day_4/.

9.2 Day 5 Final Verification & Handover Results

On Day 5, comprehensive regression testing, static analysis, and release-ready binary compilation were executed. The final verified results are documented below:

Verification Category	Command Executed	Observed Result	Quality Status
Static Code Analysis	flutter analyze	No issues found! (ran in 7.2s)	100% PASS
Unit & Model Tests	flutter test test/widget_test.dart	All 6 unit/currency tests passed	100% PASS
SQLite Database Tests	flutter test test/database_test.dart	All 6 database CRUD tests passed	100% PASS
Budget & Math Tests	flutter test test/budget_test.dart	All 15 budget calculation tests passed	100% PASS
History & Filter Tests	flutter test test/history_filter_test.dart	All 10 filter & viz tests passed	100% PASS
Total Automated Tests	flutter test	37 passed out of 37 tests (ran in 8.9s)	100% PASS
Android Binary Build	flutter build apk --debug	Built app-debug.apk (156.8 MB) in 598.8s	100% PASS
Demonstration Video	ffmpeg (1080p Full HD)	Compiled SpendWise_Week2_Demonstration.mp4	100% PASS

9.3 Automated Test Suite Breakdown

The automated test suite exercises all layers of the application stack:

- **1. Model & Currency Tests:** Validates TransactionModel and CategoryModel serialization, amount conversion to major/minor units, and CurrencyFormatter parsing rules.
- **2. Database Operations:** Verifies SQLite table creation, foreign key enforcement, newest-first ordering, parameterized updates, deletions, and SQL net balance calculation.
- **3. Budget Calculations:** Tests budget creation, edit without duplication, limit validation, remaining allowance math, 80% warning triggers, 100% over-budget alerts, and persistence after simulated restart.
- **4. History & Filters:** Verifies month filtering, category filtering, combined dual filtering, clear filters, December-to-January boundary handling, and zero-division safety on empty months.
- **5. Form Validation:** Validates form submission guards, empty amount rejection, zero and negative amount blocking, and multi-step UI CRUD workflows.

Section 10: Conclusion and Learning

The Stack & Scale Remote Internship 2026 Week 02 project—SpendWise—provided an exceptional opportunity to engineer a real-world, production-ready fintech application from concept to deployment. Through daily milestones, foundational software engineering principles were put into practice, emphasizing code clarity, architectural discipline, and rigorous verification.

10.1 Key Technical Competencies Acquired

- **1. Advanced Flutter UI/UX:** Developed proficiency in crafting custom, theme-driven fintech interfaces using Material 3 color tokens, linear gradients, dynamic elevations, and responsive layout constraints that prevent pixel overflow across compact mobile viewports.
- **2. Embedded SQLite Architecture:** Gained hands-on mastery of relational SQLite schemas, database lifecycle singletons, PRAGMA foreign_keys enforcement, automated category seeding, and parameterized queries that protect against SQL injection vulnerabilities.
- **3. Relational CRUD Engineering:** Implemented full Create, Read, Update, and Delete lifecycles across multiple relational tables with proper error handling, state synchronization, and data permanence across simulated app restarts.
- **4. Defensive Form Validation:** Designed multi-step form validation logic incorporating double-submission guards, input sanitization, type-safe currency parsing, and immediate visual user feedback.
- **5. Reactive State Management:** Constructed clean, maintainable reactive state architectures using native Flutter ChangeNotifiers, avoiding third-party library overhead while achieving instantaneous cross-screen state synchronization.
- **6. Fintech Precision Arithmetic:** Adopted the industry-standard minor currency units standard (storing integer paisas) to eliminate floating-point precision loss, implementing robust budget threshold math and zero-division safeguards.
- **7. Data Filtering & Visualization:** Architected multi-axis historical data querying, combining month navigation pills with dynamic category chips, and implemented ranked proportional progress bar visualizations.
- **8. Quality Assurance & Testing:** Cultivated disciplined testing practices by authoring 37 automated unit, widget, and database tests, guaranteeing that new feature implementations did not introduce regressions.

10.2 Artificial Intelligence Collaboration Disclosure

In accordance with internship transparency and academic integrity guidelines, artificial intelligence assistance (Antigravity by Google DeepMind) was utilized during the development of SpendWise. AI tooling was engaged as an advanced pair-programming partner for architectural planning, algorithmic review, test case design, debugging complex SQLite query interactions, and drafting technical documentation. All application source code, database structures, UI components, test assertions, and build outputs were actively verified, executed, and validated by the developer on the local development machine.

10.3 Final Concluding Statement

SpendWise successfully achieves all primary and stretch requirements established for Week 02 of the Stack & Scale Remote Internship 2026. The application stands as an exemplary demonstration of an offline-first, mathematically precise, and aesthetically refined personal finance solution. The project has profoundly expanded the developer's capabilities in mobile software engineering, database design, and automated software verification.

Documentation Sign-off

Report authored by Nida Fatima for Stack & Scale Remote Internship 2026 — Week 02. Completed and verified on 06 November 2026.